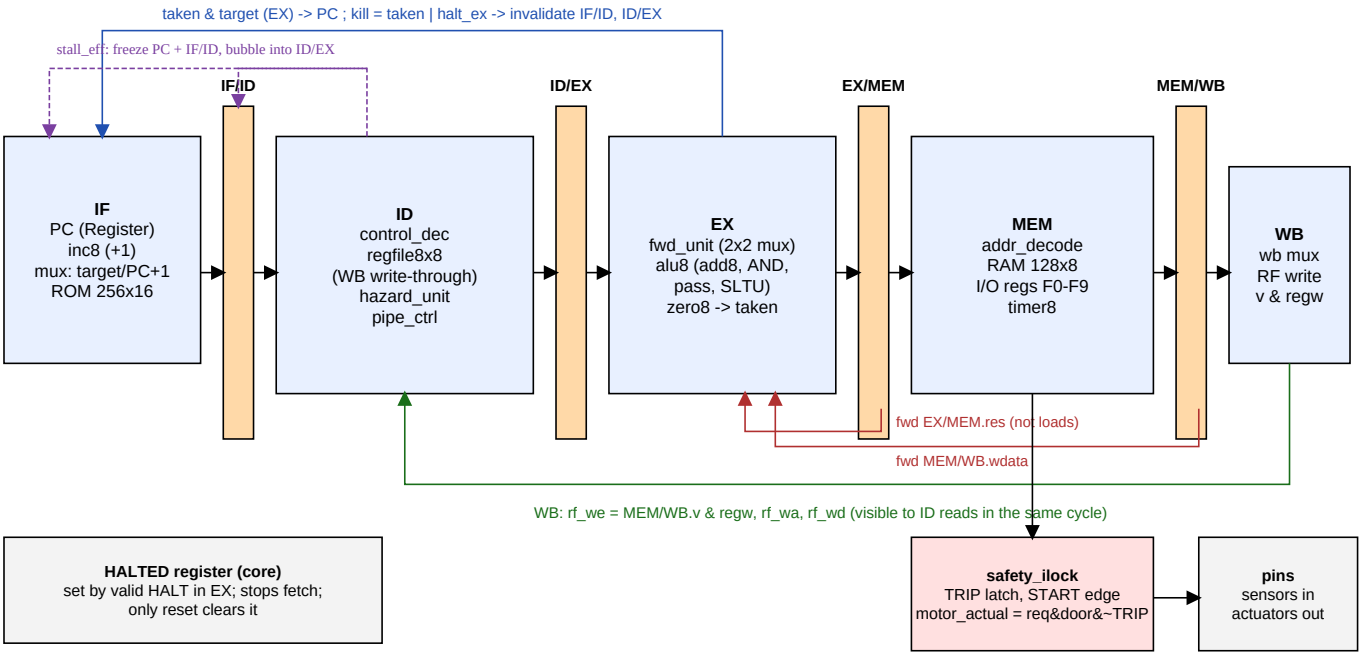
Architecture: single-issue in-order 5-stage pipeline



Hierarchy: **main** (Clock, input pins, output pins) -> **mcu_core** -> if_stage, if_id, control_dec, regfile8x8, hazard_unit, pipe_ctrl, id_ex, fwd_unit, ex_stage, ex_mem, mem_stage (addr_decode, RAM, I/O, timer8), safety_ilock, mem_wb. Arithmetic/selection blocks are own subcircuits built from AND/OR/XOR/NOT gates and splitters: full_add, half_add, add8, inc8, alu8, mux2_1/3/8, mux4_8, mux8_8, dec3to8, dec4to16, eq3, eq8, zero8. Built-ins used: Register, ROM, RAM, gates, Splitter, Tunnel, Pin, Probe, Constant, Clock, Text. One global rising-edge clock; every register uses it directly (no gated clocks); enables select hold/load. RESET drives the asynchronous clear of PC, all pipeline registers (valid=0), HALTED, ACT/DISPLAY latches, the timer and the safety latch. RAM and R0-R7 are not cleared (programs initialise them).

Pipeline-register fields (each field is one named Register, cleared by reset)

Register	Fields (width)	Bits
IF/ID (en = ~stall_eff)	v, pc[8], ir[16] ; ir = 16-bit instruction	25
ID/EX	v, pc[8], rd[3], rb[3], a[8], b[8], imm[8], aluop[2], alusub, aluimm, regw, memr, memw, beqz, bnez, jmp, halt ; rd = A-field index, rb = B-field index, a/b = RF read data, imm = IR[7:0]	50
EX/MEM	v, pc[8], rd[3], res[8], addr[8], sdata[8], regw, memr, memw ; res = ALU result, addr = forwarded B, sdata = forwarded A	39
MEM/WB	v, pc[8], rd[3], wdata[8], regw ; wdata = memr ? load data : res	21

pc is carried for observation and trace labelling only. Control-signal meanings are in isa.pdf section 2.

Hazard handling

Forwarding (fwd_unit, EX). For each operand (src = ID/EX.rd for A, ID/EX.rb for B): sel_ex = EX/MEM.v & regw & ~memr & (src == EX/MEM.rd); sel_wb = MEM/WB.v & regw & (src == MEM/WB.rd) & ~hit_ex, where hit_ex is the EX/MEM match *including* loads. So the youngest writer always wins and an older MEM/WB value can never be picked when a younger load sits in EX/MEM; the load itself is never forwarded from EX/MEM. Operand = sel_ex ? EX/MEM.res : sel_wb ? MEM/WB.wdata : RF value. Forwarded A is the store data and the branch-test value; forwarded B is the load/store address. RF write-through covers the WB-to-ID case.

Load-use (hazard_unit, ID). stall = IF/ID.v & ID/EX.v & ID/EX.memr & ((uses_a & IR[11:9] == ID/EX.rd) | (uses_b & IR[8:6] == ID/EX.rd)). **pipe_ctrl:** kill = taken | halt_ex; stall_eff = stall & ~kill (redirect wins); PC.en = taken | ~(stall_eff | halt_ex | halted); IF/ID.en = ~stall_eff; IF/ID.v_d = ~(kill | halted); ID/EX.v_d = IF/ID.v & ~(kill | stall). During the bubble PC and IF/ID hold, ID/EX gets v=0 and EX/MEM, MEM/WB keep moving. The consumer then reaches EX when the load is in MEM/WB and takes its value from there: exactly one bubble.

Branches/jumps (ex_stage). taken = ID/EX.v & (jmp | beqz & A==0 | bnez & A!=0); PC <- imm, IF/ID and ID/EX invalidated: two-instruction penalty. A flushed entry has v=0, and v gates every architectural write.

Valid gating and single commit. RF write = MEM/WB.v & regw; RAM and I/O write = EX/MEM.v & memw, decoded once in MEM and taken on the next clock edge. EX/MEM and MEM/WB are never stalled and the bubble on a stall is inserted at ID/EX, so a store spends exactly one cycle in MEM and commits at most once. Bubbles and flushed or wrong-path instructions cannot change registers, RAM or outputs.

Reset and HALT

Reset: PC=0, all valid bits 0, HALTED=0, actuator latch 0 (all requests off), DISPLAY 0, timer disabled with COUNT=LIMIT=MATCH=0, TRIP=0, START_PREV=0. HALT: when a valid HALT is in EX, halt_ex = 1. That edge invalidates IF/ID and ID/EX (younger instructions), freezes PC and sets HALTED. While HALTED=1, IF/ID keeps loading v=0, so nothing new issues. Older instructions in EX/MEM and MEM/WB complete normally. Only reset leaves the halted state. A HALT on a wrong path is invalidated by the branch flush before it reaches EX, so it never asserts halt_ex. The timer keeps counting while halted.

Safety interlock (safety_ilock) and its clocking

motor_actual = ACT.MOTOR_EN & DOOR_CLOSED & ~TRIP is purely combinational. Opening the door therefore drops the physical output immediately, with no software polling and no clock edge. TRIP is a 1-bit Register on the global clock: TRIP_d = set | (TRIP & ~rearm), set = MOTOR_EN & ~DOOR_CLOSED (door sampled open with the motor requested). START_PREV is a Register sampling START every edge, so start_rise = START & ~START_PREV is one cycle wide. rearm = TRIP & ~MOTOR_EN & DOOR_CLOSED & start_rise: software must first clear the motor request, the door must be closed, and a new 0-to-1 START edge must be accepted. set and rearm are mutually exclusive (one needs MOTOR_EN=1, the other MOTOR_EN=0), and reset clears TRIP. Closing the door alone, or a START that stays high across the door event, leaves TRIP=1. Because MOTOR_EN comes from the ACT latch, and that latch is written only by a valid store, flushed instructions cannot affect the latch either. TRIP and motor_actual are readable at IN_STAT[4], [5].